

编码规范

libterae 的 C 代码所遵循的规则, 以及每一条如何被验证

Emilio Schininà, 自动化工程硕士
版本 0.1
2026 年 11 月 6 日

版权所有 © 2026 Emilio Schininà, 自动化工程硕士。libterae 以 GNU 通用公共许可证第 3 版或任何更高版本, 或以商业许可证发布。

目录

1 编码风格 **1**
1.1 语言 . . . 1
1.2 类别与偏离 . . . 2
1.3 最影响代码形态的规则 . . . 3
1.4 版权与许可 . . . 4
1.5 文档注释 . . . 5
1.6 附加的编码风格规则 . . . 8
1.6.1 附加风格规则的其余部分 . . . 9

2 项目规则 **11**
2.1 代码放在哪里 . . . 11
2.2 代码可以调用什么 . . . 13
2.3 程序与测试 . . . 14
2.4 协议行为 . . . 14
2.5 来自别处的文本 . . . 15
2.6 与检查协同工作 . . . 15

3 自动验证 **17**
3.1 自动检查的种类 . . . 17
3.2 验证一项修改 . . . 18
3.3 检查自身的规则 . . . 18

4 MISRA C:2012 逐条规范 **21**
4.1 怎样读一行 . . . 21
4.2 指令 . . . 21
4.3 语言标准 . . . 23
4.4 未使用的代码与死代码 . . . 24

coding-guidelines-zh.tex Page iv November 8, 2026 13:15

4.5 注释 . . . 25
4.6 字符与转义序列 . . . 25
4.7 标识符 . . . 25
4.8 位域 . . . 26
4.9 字面量与常量 . . . 27
4.10 声明与定义 . . . 27
4.11 初始化 . . . 28
4.12 本质类型 . . . 29
4.13 指针转换 . . . 30
4.14 表达式 . . . 31
4.15 副作用 . . . 32
4.16 控制表达式 . . . 32
4.17 控制流 . . . 33
4.18 switch 语句 . . . 34
4.19 函数 . . . 34
4.20 指针与数组 . . . 35
4.21 重叠的存储 . . . 36
4.22 预处理器 . . . 37
4.23 标准库 . . . 38
4.24 资源与线程 . . . 40
4.25 泛型选择 . . . 42

5 覆盖率一览 45

5.1 按验证方式统计的规范 . . . 45
5.2 已知的漏检 . . . 45
5.3 人工审阅什么 . . . 46
5.4 只有人工才验证什么 . . . 46

1

编码风格

这些规范是 libterae 的 C 代码从设计、开发到维护所遵循的一组规则。每条规则都说明它要求什么，以及如何被验证：由自动检查验证、由审阅代码的人验证，或由两者共同验证——自动检查发现一部分违规，审阅覆盖其余部分。规则用什么来验证、为了什么目的验证，从来不属于规则本身。

本文档分为五章：

- **第 1 章，编码风格**。给出所用的语言及其标准、MISRA 的类别以及如何记录对某条规则的偏离、最影响代码形态的规范、版权与许可、文档注释，以及附加的编码风格规则。
- **第 2 章，项目规则**。说明代码放在哪里、可以调用什么、程序与测试、协议行为、来自别处的文本，以及一项修改在被称为完成之前如何被验证。
- **第 3 章，自动验证**。列出项目开发的各类自动检查、一项修改用什么来验证，以及检查本身所遵循的规则。
- **第 4 章，MISRA C:2012 逐条规范**。用项目自己的措辞逐条列出全部 221 条规范，每条给出其类别、如何被验证，以及什么能表明该验证有效。
- **第 5 章，覆盖率一览**。按验证方式统计各条规范、还需要人工审阅什么，以及已知会被检查漏掉的违规。

在表格中，最后一列说明一条规则如何被验证。自动表示某项检查会报告该违规并指出所违反的规则。人工表示必须由审阅者验证。部分表示检查能发现一部分违规，其余仍需人工审阅。

1.1 语言

libterae 用 C99 编写，并遵循 MISRA C:2012。它所受约束的规范是 2013 版加上第 1、2 号修正案：175 条规范。第 3、4 号修正案又增加 46 条，这些同样被遵循，违反时与其它条

coding-guidelines-zh.tex Page 2 November 8, 2026 13:15

款一样报告，但项目的声明只针对第 1、2 号修正案，第 5 章把后来的条款单独统计，作为跟踪项。在那 46 条中，29 条涉及 C11 特性，C99 代码无法使用，因此不会出现。

库由 `src/` 与 `include/` 组成：它采用 MISRA C:2012，含第 1 至 4 号修正案。`platform/` 不在 MISRA 声明的范围内。

表 1.1 语言

规则	含义	验证方式
C99	代码是 C99，即项目所用的语言方言，不使用任何编译器扩展，也不使用 C11 的任何内容。	自动：代码以 `-std=c99 -pedantic-errors` 编译，C11 的线程与原子头文件被拒绝。
MISRA	遵循带第 1 至 4 号修正案的 MISRA C:2012，逐条照第 4 章所列执行。	自动：221 条规范中的 184 条，其中 47 条为部分验证。 人工：6 条。 第 5 章给出统计。
合规性	检查全部通过并不等于声明合规。部分验证或完全未验证的规范仍需审阅。	人工。

1.2 类别与偏离

每条 MISRA 规范都有一个类别，说明它在这里被遵循的严格程度：

表 1.2 本项目中 MISRA 的类别

强制	始终遵循，不允许偏离。
必需	遵循，除非记录了偏离。
建议	与必需同等对待。

偏离是针对某条必需或建议规则所记录的例外，用于确实无法合理遵循的情形。MISRA 发布了一些偏离许可，已经覆盖若干常见情形，主要针对生成的代码；条目可以引用其中之一，但多数并不适用于人编写和审阅的代码。

coding-guidelines-zh.tex Page 3 November 8, 2026 13:15

表 1.3 偏离

规则	含义	验证方式
一份清单	每个例外都是同一份清单中的一条，所有检查都读取这份清单。代码中的内联抑制被关闭，以便例外全部留在清单里。不带清单运行不算通过。	自动。
条目的内容	条目按报告打印的形式给出检查名，以及文件和行号，例如 `Wconversion:*/src/example.c:42`。	自动。
理由	条目说明该例外为何安全、由谁批准、何时批准。	人工。
批准	新的偏离需要有人批准，且在加入之前提出申请。	人工。
强制规则	强制规则永远不允许偏离。	人工。 只有清单顶部的注释说明这一点。
许可	条目可以引用一项许可，但仍需有自己的理由和批准人。	人工。
错误	未指明检查名的错误，例如语法错误，不能偏离。	自动。

1.3 最影响代码形态的规则

以下规范决定了 libterae 代码外观的大部分。其余的在第 4 章。

表 1.4 最影响代码形态的规则

规则	含义	验证方式
D4.12、21.3	不准动态内存：不使用 `malloc`、`free` 或类似的函数。	自动：规则 21.3 完全验证，指令 4.12 部分验证。
21.6	不使用 `<stdio.h>` 的输入输出。	自动。

续页

coding-guidelines-zh.tex Page 4 November 8, 2026 13:15

表 1.4 最影响代码形态的规则(续)

规则	含义	验证方式
17.2	不准递归。	自动,部分验证。 通过函数指针的递归是已知的漏检项。
15.1	不准使用 `goto`。	自动。 标号也被拒绝,因此 *goto* 完全不可能出现。
15.5	每个函数只有一个出口。	自动。
15.6	每个 `if`、`else` 和循环体都要加花括号。	自动。
16.4	每个 `switch` 都有 `default`。	自动。
D4.6	使用说明了宽度的类型,如 `uint32_t`,而不是裸 `int`。	自动,部分验证。
D4.10	每个头文件都有包含守卫,命名为 `MQTT_<FILE NAME>_H`。	自动。
D4.14	从程序外部收到的每个值都要检查,包括每一个 MQTT 报文。	人工。 测试和模糊测试目标会尝试,但无法证明。

还有一条规则影响接口,虽然没有哪条准则要求它。接口的函数在通过任何指针读写之前,先检查给它的每个指针。只有在函数文档说可以的地方,或者指针带着为零的长度时,指针才可以是 NULL。在其他地方,调用被拒绝:它不改变任何东西,返回与对象没有初始化时同一调用所返回的值。这超出了指令 4.14 的范围,后者针对的是来自程序外部的值:这条规则把调用方的错误变成一个错误结果,而不是崩溃。没有工具检查这一点,所以每个这样的函数都有一个测试,依次给它的每个指针传入 NULL。

1.4 版权与许可

libterae 同时以 GPL 和商业许可提供,因此其全部代码必须能在两种许可下发布。

coding-guidelines-zh.tex Page 5 November 8, 2026 13:15

表 1.5 版权与许可

规则	含义	验证方式
声明	每个 .c 和 .h 文件都以一段包含版权声明的注释开头，位于其它一切之前。	自动，覆盖每个文件夹。
归属	声明是正确的，归属于正确的所有者。	人工。 通过只表明存在版权声明。
引用	引用某份文档的注释给出其标题和章节，而不给出其版权行，否则会被读成代码的版权。注释用它自己的话说明该文档所说。	人工。
来自别处的代码	保留其版权和许可声明，且只有在其许可证允许以两种许可发布时才使用。MIT 和 BSD 代码通常可以；他人的 GPL 代码不行，除非其版权持有人同意。	人工。 *src/port/*的每个文件夹都有 *NOTICE*，说明其中文件的来源及其许可证。
贡献	在合并他人的任何内容之前，哪怕是很小的修复，对方都要签署一份协议，授予重新许可的权利。	人工。

1.5 文档注释

MISRA C:2012 不规定哪些东西需要文档、文档写在哪里、包含什么。它只在规则 3.1、3.2 和指令 4.4 中限制注释内部可以放什么。因此这些规则是项目自己的。它们很严格：任何地方只用一种块形式。与 MISRA 的规则不同，它们没有偏离，发现的问题也不能被抑制。

每一个被命名的东西都需要一个文档块：宏、函数、typedef、标签、枚举常量、struct 与 union 成员，以及所有作用域中的对象，包括局部变量和 `for` 循环变量。参数在所属函数的块中记录。唯一的例外是包含守卫 `MQTT_<FILE NAME>_H`，当它是头文件中的第一项时：

```
/**
 * @file base_binary_t.h
 * @brief Binary Data.
 *
 * On the wire, a 16-bit count of the data bytes comes first and the data
 * bytes come after it.
 *
 * [V5] MQTT Version 5.0, OASIS Standard, 7 March 2019, 137 pages.
 * @li Cited: 1.5.6, p.19.
 * @li Local: the specification, kept with the project.
 * @li Source: docs.oasis-open.org/mqtt/mqtt/v5.0/os/
```

coding-guidelines-zh.tex Page 6 November 8, 2026 13:15


```
 */
#ifndef MQTT_BASE_BINARY_T_H
#define MQTT_BASE_BINARY_T_H

/** @brief Reference to binary data. It does not own the bytes. */
typedef struct {
    /** @brief Number of bytes in data. */
    uint16_t length;
} mqtt_binary;

/**
 * @brief Report whether two strings differ.
 * @param[in] actual String under test.
 * @param[in] expected String to compare with.
 * @return 1U if they differ, otherwise 0U.
 */
static uint32_t defs_text_differs(const char *actual, const char *expected);
```

表 1.6 文档注释

规则	含义	验证方式
DOC-1	`/** */` 是唯一的形式: 写在一行上, 或写成 `/**`、若干 `* text` 行, 然后 `*/`。`/*!`、`/***`、`/**<`、`///` 和 `//!` 都不合格。	自动。
DOC-2	块紧贴在被记录对象的第一个记号上方, 缩进相同, 中间不隔任何东西。`for` 循环变量的块放在 `for` 上方。	自动。
DOC-3	每个东西有一个块, 每个块属于一个东西。函数或对象只在首次声明处记录, 头文件先于 C 文件。此后的声明不带块, 并使用相同的参数名。	自动。
DOC-4	文件以其版权注释开头, 接着是一个以 `@file` 及其准确文件名开头的块, 位于任何代码之前。	自动。
DOC-5	块以恰好一个 `@brief` 开头; 在文件自身的块中, 它位于 `@file` 之后。只允许这些标签: `@file`、`@brief`、`@param`、`@return`、`@retval`、`@li`, 以及正文中的 `@c` 和 `@p`。`@li` 跟在段落或另一个 `@li` 之后。不许有 `\` 命令, 也不许有 `@code`, 那会与指令 4.4 冲突。	自动。

续页

coding-guidelines-zh.tex Page 7 November 8, 2026 13:15

表 1.6 文档注释(续)

规则	含义	验证方式
DOC-6	函数、类函数宏和函数指针 typedef 为每个参数按顺序写一条 `@param[in|out|in,out] name Text.`。函数若返回值, 写一条 `@return`, 否则不写。宏上它是可选的。	自动。
DOC-7	正文是 ASCII, 不超出第 80 列(STYLE-1), 并以句号结尾, `@li` 行除外。正文不得含 \、<、&、# 或 %, 也不得在字母前出现 $, 每行的 " 必须成对: 否则读注释的工具会把它们当成命令。	自动。
DOC-8	其它注释用 `/* */`, 且只用于版权、`#endif` 旁、函数体内或初始化器中。它们不得直接位于需要块的东西上方。不允许 `//` 注释。	自动。
DOC-9	代码必须是检查能完整读取的形式。其它一切都是发现项, 绝不跳过。	自动。
真实性	注释真实、有用、保持最新, 代码可以追溯到它的用途, 正如指令 3.1 所要求。	人工。 通过只表明每条注释存在、位置正确、格式合规。

DOC-9 排除了一些合法的 C, 因为下列每一项都可能对检查隐藏某些东西:

- 一条声明里多于一个对象, 如 `int a, *b;`。
- 函数指针, 除非写成 `typedef R (*name)(params);`。
- struct、union 或 enum 的体, 若不是恰好写成 `typedef struct { ... } name;` 或 `struct name { ... };`, 就不合格。这排除了 `typedef struct x { ... } x;`、`struct { ... } x;` 和嵌套的体。不透明类型可写成 `typedef struct x x;`, 再在 C 文件中定义 `struct x`。
- 单独的 `struct x;`, 以及从未被定义或 typedef 的标签。
- `sizeof`、强制转换、复合字面量或参数中的类型体。
- `f()`、K&R 定义、`...` 和无名参数。
- `#undef`、`#if 0`、`#line`、`#pragma`、`_Pragma`、`asm`、`__attribute__`、`goto`、标号、三字符组和二字符组。行拼接只允许出现在 `#define` 中。
- `#if` 与其 `#else`、`#endif` 处于不同的嵌套层级, 或位于一条声明内部。
- 位于代码之后的 `#include`, 所包含的不是直接写出的 `.h` 文件, 或包含的是检查未

coding-guidelines-zh.tex Page 8 November 8, 2026 13:15

列出的系统头文件。MISRA 禁止某些系统头文件，因此它们未被列出。

- 宏体含 ;、{ 或 }、存储类、typedef、struct、union、enum 或 ##，或带有括号之外的类型或参数。这排除了 do { ... } while (0) 宏：请写函数，正如指令 4.9 所主张的。

1.6 附加的编码风格规则

以下是附加的编码风格规则，部分采纳自德州仪器的《推荐 C 风格与编码指南》。它们适用于 include/、platform/ 和 src/ 中每个 .c 与 .h 文件，以及测试。与文档规则一样，它们没有偏离。表中第二列给出该规则在那份文档中的编号，供参考。

表 1.7 附加的编码风格规则

规则	TI	含义	验证方式
STYLE-1	R1, R2	任何一行都不得越过第 80 列，注释也算。放不下的语句在逗号、运算符或左括号之后换行：绝不在运算符之前，也绝不在类型与其所声明的名字之间。相邻的字符串字面量可以分行。	自动。
STYLE-2	R3	用空格缩进，绝不用制表符：每层 4 个空格，从第 1 列算起。花括号内的内容比花括号深一层，case 和 default 与其 switch 同层。续行可以与它所延续的部分对齐。位于行首的注释从 4 的倍数开始。	自动。
STYLE-3	R4	不得连续两个空行，文件末尾不得有空行，文件以换行结束。块内的各个步骤之间用一个空行分隔。	自动。 块内步骤之间是否有空行由人工判断。
STYLE-4	R5	二元运算符两侧各一个空格，包括 =、复合赋值和 ? :。一元运算符后不加空格（!x、~x、-x、*p、&x），-> 和 . 两侧不加空格，(与 [之后、) 与] 之前不加空格，名字与其 (之间也不加。逗号或分号之后一个空格，之前没有。if、for、while、switch 或 return 与其 (之间一个空格，sizeof 之后没有。	自动。

续页

coding-guidelines-zh.tex Page 9 November 8, 2026 13:15

表 1.7 附加的编码风格规则(续)

规则	TI	含义	验证方式
STYLE-5	R9	不用 ++ 或 --:写成 `x += 1U`。TI 允许它们在语句中单独出现, libterae 完全不用。	自动。
STYLE-6	R45	文件的顶层部分按此顺序排列:include、宏、类型、对象、函数声明、函数定义。版权注释和 `@file` 块在它们之前(DOC-4),包含守卫把它们围在中间。	自动。
STYLE-7	G15	一个文件最多 2,000 行。更长的拆分成文件,每个文件承担这项工作的一部分。	自动。
STYLE-8	G16	`src/` 或 `platform/` 中的函数最多 50 个可执行行:其函数体中含代码的行,不含只有花括号的行和不产生值的声明。	自动。 测试不受此限。

1.6.1 附加风格规则的其余部分

那份文档中其余规则,或在别处已覆盖,或未被采纳:

- **由 MISRA C:2012 覆盖。**R8(12.1)、R11(D4.4)、R14(D4.10)、R19、R31、R32 和 R36(8.1 至 8.8、17.3)、R22(11.9)、R44(15.6)、G17(15.5)、G20(9.1)和 G21(13.4)。该文档第 2 节列出了它遵循的 MISRA C:2004 规则;libterae 改为遵循 MISRA C:2012。
- **由文档规则和版权声明覆盖。**R6、R7、R13、R16、R41、R46 和 G10。
- **由项目规则覆盖。**G1 和 G18,见第 2 章。
- **未采纳。**命名(R17、R18、R20、R21、R34、R35、R39、R40、G6 至 G9、G11 至 G13)、文件名(R23 至 R28)、花括号位置(R43)、注释(R12、G3、G4)、文档块内容(R29、R38、R42、G14)、修订历史(G2)、注释密度(G5)、额外变量(G19)和头文件(R10、R15、R30、R33 和 R37)。这些由 libterae 自己的规则决定,因此其名字保持 `lower_snake_case`,控制语句的开花括号留在同一行。

2

项目规则

这些规则说明代码放在哪里、可以调用什么、程序与测试如何组织、协议行为如何取自规范，以及一项修改在被称为完成之前如何被验证。

2.1 代码放在哪里

项目就是库。服务器和客户端是表明它能工作的测试，不是产品。库是 src/ 与 include/，platform/ 是与操作系统打交道的部分。

表 2.1 文件夹

路径	内容
include/base_*.h	库：各版本共用的定义，使用 MQTT_ 和 mqtt_ 前缀。
include/v31_*.h、v311_*.h、v5_*.h	库：某一版本的定义，使用 MQTT31_、MQTT311_ 和 MQTT5_ 前缀。
src/	库：实现，不含系统调用、不含分配器，也不用 C 库。
src/port/{architecture}/	库：编译器生成的代码所调用的内存函数，每种架构一个文件夹：aarch64、arm、armv4t、armv7m、avr、i386、m68k、mips、powerpc64、riscv32、riscv64、rx、sh、x86_64 和 xtensa。每个都有一份 NOTICE，说明其中文件的来源及其许可证。
src/port/generic/	库：C 版的 memcpy、memmove、memset 和 memcmp，供没有相应汇编的架构使用，也供没有其它文件夹为其编写的处理器使用。某个文件夹对某个函数没有汇编实现时，就包含这里的一个文件。

续页

coding-guidelines-zh.tex Page 12 November 8, 2026 13:15

表 2.1 文件夹(续)

路径	内容
`platform/`	受支持系统的套接字、时钟与 TLS：POSIX、Linux、macOS 和 Windows，以及实时系统 FreeRTOS 和 Micrium 的 uC/OS-III。不在 MISRA 声明范围内。
测试	检查每个定义的一个测试、每个模块一个测试、基于本库构建的一个服务器和一个客户端、一个负载生成器，以及模糊测试目标。

表 2.2 代码放在哪里

规则	含义	验证方式
库	所有协议策略和算法都属于库。服务器和客户端只调用它的接口，自己不持有任何决策或算法。若其中一方需要库没有提供的东西，就把它加到库里。	人工。
扁平文件夹	`src/` 和 `include/` 没有子文件夹，只有 `src/port/{architecture}/` 例外。	人工。
port 文件夹	`src/port/{architecture}/` 存放编译器调用的内存函数，能用其它项目的汇编就用汇编，不能就用 `src/port/generic/` 里的 C。来自其它项目的内容逐字保留。这些文件夹不在 C 检查范围内。	自动：每个 `port_*.S` 只定义那四个内存函数和 C 保留的名字，且不留下未定义符号。人工：检查每个文件夹的 `NOTICE`。
通用 C	`src/port/generic/` 有 `memcpy`、`memmove`、`memset` 和 `memcmp` 的 C 版本，它们绝不调用这四个内存函数中的任何一个。	自动，在每个优化级别下。
前缀	`base_` 文件中的名字以 `MQTT_` 或 `mqtt_` 开头，某一版本文件中的名字以 `MQTT31_`、`MQTT311_` 或 `MQTT5_` 开头。	人工。*`MQTT_<FILE NAME>_H`* 是唯一允许不带块的包含守卫。
命名	名字取自规范自身的术语，如 Packet Identifier、Fixed Header 或 Receive Maximum，而不是自造的名字或缩写。	人工。

coding-guidelines-zh.tex Page 13 November 8, 2026 13:15

2.2 代码可以调用什么

表 2.3 代码可以调用什么

规则	含义	验证方式
不用 C 库	`src/` 不含系统调用、不含分配器，也不用 C 库的任何东西：它调用的只有编译器辅助函数。	自动：在每个目标上，在 `-O0`、`-O1`、`-O2` 和 `-Os` 下验证，规则 21.3 和指令 4.12 拒绝分配器。构建使用 `-ffreestanding`。
每个系统一份清单	代码从库外取得的，只有其系统清单允许的内容，而这些清单对应 `platform/` 各文件夹所面向的系统：`posix/` 用 C99 和 POSIX.1-2008，`linux/` 加上 Linux 的补充，`macos/` 加上 macOS 的补充，`win32/` 用 Winsock 2 和 Windows API，`freertos/` 用 FreeRTOS 和 FreeRTOS+TCP，`ucos3/` 用 uC/OS-III 和 uC/TCP-IP。不因速度而有例外。	自动：把每个目标文件所调用的符号与其系统清单比较，只有连同所需理由一起才能加入新名字。
POSIX 的基础	文件夹若以 POSIX 为基础，则遵循 POSIX.1-2008 的基础部分。POSIX 的可选部分，或者 POSIX 没有提到的文件，只在基础部分无法完成任务时才使用，而每一处这样的使用都是写明理由的例外。有自己做法的系统遵循自己的清单：`linux/` 用 epoll 等待而不是 `poll()`，`macos/` 用 kqueue。	自动：`posix/` 编译时只定义 `_POSIX_C_SOURCE`，POSIX 没有的函数因此未声明，而未声明即错误；其所调用的再与清单比较，清单中每个例外都附有理由。
系统头文件	只有某个系统才定义的系统头文件，只能在该系统的文件夹中加入，且只有 `platform/` 包含操作系统和 TLS 库的头文件。	自动：不在该文件夹清单内的头文件即发现项，且 `src/` 不能包含 `platform/` 的头文件。
`platform/`	`platform/` 不在 MISRA 声明范围内，因为项目对那部分代码自身的分析尚不完整。	人工：它的 MISRA 发现项会被报告，由审阅者阅读，但不使一次运行失败。

coding-guidelines-zh.tex Page 14 November 8, 2026 13:15

2.3 程序与测试

测试保存在项目的测试与验证文件夹中。它们不是产品：它们的存在是为了表明库能工作。

表 2.4 程序与测试

规则	含义	验证方式
受保护的 `main`	每个程序的 `main` 都由某个宏守护，如 `SERVER_PROGRAM`，因为 MISRA 把所有程序当成一个来读。新程序还要在测试运行器中登记。	人工。
每个定义	`include/` 中的每个宏都被检查定义的那个测试使用，该测试验证它的值。	自动：规则 2.5 报告无人使用的宏，定义测试在值不对时以非零状态退出。
模块测试	每个模块在测试文件夹中有一个测试，它们全都在 AddressSanitizer 和 UndefinedBehaviorSanitizer 下通过。	自动，针对净化器。 人工，针对每个模块有一个测试。

2.4 协议行为

表 2.5 协议行为

规则	含义	验证方式
来源	协议行为取自规范，而不是凭记忆，且文件的 `@file` 块引用相应章节。	人工。 通过只表明块存在。
每条规则	MQTT 3.1.1 和 5.0 的每条编号规则，以及 MQTT 3.1 所要求事项清单中的每一项，都由某个测试引用，否则理由文件说明为何没有适用的测试。	自动。 引用记录的是一种确信：它不能证明测试是对的。
与其它实现互通	改动协议行为之后，服务器和客户端仍能与其它 MQTT 实现协同工作、通过那些项目自己的测试，并经受住模糊测试。	自动，一次运行一类。

续页

coding-guidelines-zh.tex Page 15 November 8, 2026 13:15

表 2.5 协议行为(续)

规则	含义	验证方式
决定	规范留有选择余地的地方, 所选的做法记录在项目的设计笔记中。	人工。

2.5 来自别处的文本

表 2.6 来自别处的文本

规则	含义	验证方式
自己的话	MISRA 的文本和规范的文本绝不粘贴进项目: 要用项目自己的话描述。	人工。
规则只有一个来源	每条 MISRA 规范用 libterae 自己的话写成的描述只保存在一处, 其它每份副本都据它写出。规则的增加或修改都在那里进行, 绝不在副本中进行。	自动: 把各副本与来源比较, 每个示例都跑一遍检查, 违规的必须被报告, 合规的版本必须不被报告。

2.6 与检查协同工作

表 2.7 与检查协同工作

规则	含义	验证方式
完成之前	一项 C 修改, 只有适用于它的每项检查都运行过且未发现问题, 才算完成。什么也没检查的运行不算通过, 读到代码之前就停下的检查也不算通过。	人工: 没有任何东西自动运行这些检查。
修改之后	改动 `src/` 或 `platform/` 之后, 重跑针对代码和针对代码可调用内容的检查。改动协议行为之后, 重跑与其它实现对照的测试和模糊测试器。改动 `include/` 或 `src/` 中的注释之后, 重新检查文档规则。	人工。

续页

coding-guidelines-zh.tex Page 16 November 8, 2026 13:15

表 2.7 与检查协同工作(续)

规则	含义	验证方式
改代码	发现项在代码中修正。绝不为了通过而修改检查。	人工。
检查的测试	改动某个检查之后,运行它自己的测试。	人工。

3

自动验证

这些规范并不取决于它们如何被验证，但项目开发自己的外部工具来检查它们是否被遵守。每个工具覆盖一类规则，读取代码、构建或测试。它们在一项修改被称为完成之前由人运行，没有任何东西自动运行它们。

以下的说明是刻意笼统的：规则规定一项检查必须发现什么，而工具只要发现的东西不变，就可以自由更换。

3.1 自动检查的种类

检查有以下几种，一条规则由其中适用的一种或几种来验证：

表 3.1 自动检查的种类

种类	读取什么
源文本	按写出的样子读取文件，不做预处理，因此条件编译的每个分支都被读到。它发现文本自身就能说明的东西：注释的形态、缩进、行长、文件各部分的顺序、包含守卫和版权声明。
编译后的程序	编译器构建出的代码：其真实类型（穿透每一层 typedef 和宏），以及项目要求的所有警告，全部按错误处理。
语法树分析	编译器眼中的代码，而不运行它：类型转换、声明、预处理器、标准库，以及需要一次知道整个程序才能判断的规则。
值与路径分析	某个变量在某一行可能持有什么值，函数中有哪些路径可达：未使用的值、未初始化的读取、越界访问、除以零和缺少返回。

续页

coding-guidelines-zh.tex Page 18 November 8, 2026 13:15

表 3.1 自动检查的种类(续)

种类	读取什么
目标文件调用了什么	每个目标文件引用的符号, 与它所面向系统的清单比较, 覆盖每个受支持的目标和处理器。
测试	库运行时的行为: 定义测试、每个模块在两个净化器下的测试、被测试引用的协议规则、其它实现编写的测试, 以及模糊测试目标。
检查本身	检查自己的测试, 表明一项检查在应当报告时报告, 在不应报告时保持沉默。从未被看到触发过的检查不予计入。

3.2 验证一项修改

一项 C 修改,只有适用于它的每项检查都运行到底且未发现问题,才算完成。因为什么也没读所以什么也没发现的检查,或在读到代码之前就停下的检查,不算通过。

表 3.2 一项修改用什么来验证

修改所在	需要哪些检查
`src/` 或 `include/`	源文本检查、构建检查、语法树检查和值检查, 以及检查定义的那个测试。
`platform/`	同上, 另加一项检查: 目标文件所调用的是否在该系统的清单上, 对每个系统、每种处理器。
协议行为	引用协议规则的测试、其它实现编写的测试, 以及模糊测试目标, 一次运行一类。
某项检查	该检查自己的测试。

3.3 检查自身的规则

检查是项目的一部分,遵循以下规则:

coding-guidelines-zh.tex Page 19 November 8, 2026 13:15

表 3.3 检查自身的规则

规则	含义	验证方式
一个发现项对一条规则	发现项指出它违反了哪条规则，以便查阅、修正，必要时记录偏离。	自动：不带规则的发现项是检查本身的失败。
不悄悄跳过	检查无法归类的代码是发现项，绝不是它跳过的东西。	自动。
什么也没发现不等于通过	检查要报告它什么也没读到，而那不算通过。	自动。
证明会触发	每项检查都要证明：在违反规则的代码上触发，在遵守规则的代码上保持沉默。	人工，针对所选的用例。 自动，针对这些用例的运行。
一份例外清单	每个偏离都记录在同一份清单中，由检查读取，代码中不持有自己的抑制。	自动。
改代码	发现项在代码中修正，绝不通过修改检查来消除。	人工。

4

MISRA C:2012 逐条规范

本章列出 MISRA C:2012 的全部 221 条规范:2013 版、第 1 至 4 号修正案和技术勘误 2。每一行用 libterae 自己的话说明该规范要求什么,以及它如何被验证。准确的措辞属于 MISRA,见 MISRA 发布的规范文件。

4.1 怎样读一行

每条规范的编号下方是它的类别。最后一列说明该规范如何被验证:

- **自动**。某项检查会报告违规,并指出所违反的规范。
- **人工**。没有检查覆盖该规范,因此必须由审阅者验证。这些规范取决于程序运行时做什么,或取决于意图。

这两个词下面可能带有附注:

- **部分**。检查能发现部分违规,其余仍由人工审阅。该规范取决于程序运行时做什么,或取决于意图。
- **第 3、4 号修正案**。与其它条款一样验证,但在项目的声明范围之外,如第 1.1 节所述。
- **在 C99 中不会出现**。该规范涉及 C11 的特性,C99 代码无法使用。构建按 C99 予以拒绝。

第 5 章对此作出统计,并列出检查已知会漏掉的违规形式。

4.2 指令

指令针对整个代码。指令 1 关于实现,指令 2 关于编译,指令 3 关于需求,指令 4 关于代码设计,指令 5 关于并发。

coding-guidelines-zh.tex Page 22 November 8, 2026 13:15

表 4.1 指令 1.1 至 5.3

规范	要求什么	验证方式
Dir 1.1 必需	程序输出若取决于 C 标准留给编译器决定的事项, 项目记录下本编译器的做法。	人工。
Dir 2.1 必需	每个源文件都编译通过, 没有一个错误。	自动。
Dir 3.1 必需	每一段代码存在都是因为某项有文档的需求要求它, 且可以追溯到那项需求。	人工。
Dir 4.1 必需	代码的写法使运行时的失败不会发生, 例如除零、溢出、下标越界或空指针, 并对可能引发这些的值进行检查。	自动。 部分。
Dir 4.2 建议	每一处汇编语言的使用, 都在出现处加以说明。	人工。
Dir 4.3 必需	汇编语言与 C 代码分开, 只放在不含其它内容的函数或宏中。	自动。 部分。
Dir 4.4 建议	不再需要的代码直接删除, 不留在注释里。	自动。 部分。
Dir 4.5 建议	在同一处可见、且处于同一名字空间的两个名字不得相像: 它们之间的差别不止是大小写, 也不止是 `l` 与 `1`、或 `O` 与 `0` 这样的字符。	自动。 部分。
Dir 4.6 建议	数值使用说明了宽度与符号性的 typedef, 如 `uint32_t`, 而不是宽度随机器而定的基本类型。	自动。 部分。
Dir 4.7 必需	函数报告错误时, 调用者检验它所报告的内容。	自动。 部分。
Dir 4.8 建议	一个文件若只是传递指向某个结构体或联合体的指针, 从不查看其内部, 就不让它看到该结构体包含什么: 它看到的是一个不完整的声明。	自动。 部分。
Dir 4.9 建议	函数能完成与类函数宏相同的工作时, 用函数。	自动。 部分。
Dir 4.10 必需	每个头文件都保护自己不被重复包含, 例如用包含守卫把全部内容围起来。	自动。

续页

coding-guidelines-zh.tex Page 23 November 8, 2026 13:15

表 4.1 指令 1.1 至 5.3(续)

规范	要求什么	验证方式
Dir 4.11 必需	传给库函数的值, 若该函数对某些值的结果未定义, 则先检查再传。	自动。 部分。
Dir 4.12 必需	程序运行期间绝不分配或释放内存: 没有堆。	自动。 部分。
Dir 4.13 建议	作用于同一资源的函数, 按该资源所需的顺序调用: 使用之前打开, 关闭之后不再使用。	自动。 部分。
Dir 4.14 必需	来自程序外部的每个值, 例如收到的报文字节, 都在使用之前检查。	人工。
Dir 4.15 必需	可能产生无穷大或 NaN 的浮点计算, 其结果在进入未准备处理它们的代码之前先被检验。	人工。 第 *3*、*4* 号修正案。
Dir 5.1 必需	线程之间不发生竞态: 两个线程共用的数据受到保护, 使一个线程写入时另一个不能读或写。	自动。 部分。 第 *3*、*4* 号修正案。
Dir 5.2 必需	线程不会死锁: 加锁总是按同一顺序, 因此不会出现两个线程互相等待对方持有的锁。	人工。 第 *3*、*4* 号修正案。
Dir 5.3 必需	线程只在程序启动时创建, 绝不在之后的运行期间创建。	自动。 部分。 第 *3*、*4* 号修正案。

4.3 语言标准

表 4.2 规则 1.1 至 1.5

规范	要求什么	验证方式
Rule 1.1 必需	代码是标准 C: 不违反语言的任何语法规则或约束, 也不超出每个编译器都必须支持的翻译限制。	自动。
Rule 1.2 建议	代码不使用编译器对语言自行添加的东西。	自动。

续页

coding-guidelines-zh.tex Page 24 November 8, 2026 13:15

表 4.2 规则 1.1 至 1.5(续)

规范	要求什么	验证方式
Rule 1.3 必需	程序中没有任何未定义行为, 也没有会改变程序行为的未指明行为。	自动。 部分。
Rule 1.4 必需	不使用 C11 附录 K 中可选的边界检查函数, 如 `memcpy_s`。第 3、4 号修正案把其它的 C11 特性从本条移出, 各给了它们自己的规范。	自动。
Rule 1.5 必需	不使用 C 标准标记为已过时的特性, 例如类型之后的存储类说明符、以空括号声明的函数, 或单独列出参数类型的定义。	自动。 第 *3*、*4* 号修正案。

4.4 未使用的代码与死代码

表 4.3 规则 2.1 至 2.8

规范	要求什么	验证方式
Rule 2.1 必需	没有永远无法执行的代码。	自动。 部分。
Rule 2.2 必需	没有死代码: 不执行任何一旦删除、程序行为仍相同的代码。	自动。 部分。
Rule 2.3 建议	声明的每个类型都在某处被使用。	自动。
Rule 2.4 建议	声明的每个结构体、联合体或枚举标签都在某处被使用。	自动。
Rule 2.5 建议	定义的每个宏都在某处被使用。	自动。
Rule 2.6 建议	函数中没有不被任何 `goto` 跳转到的标号。	自动。
Rule 2.7 建议	函数使用它的每一个参数。	自动。
Rule 2.8 建议	定义的每个对象都在某处被使用。	自动。 第 *3*、*4* 号修正案。

coding-guidelines-zh.tex Page 25 November 8, 2026 13:15

4.5 注释

表 4.4 规则 3.1 至 3.2

规范	要求什么	验证方式
Rule 3.1 必需	注释中不含另一个注释的开头, 即 `/*` 或 `//`, 但网址例外, 如 `https://example.org`。	自动。
Rule 3.2 必需	`//` 注释不以反斜杠结尾, 否则下一行会被并入注释。	自动。

4.6 字符与转义序列

表 4.5 规则 4.1 至 4.2

规范	要求什么	验证方式
Rule 4.1 必需	字符串或字符常量中的八进制或十六进制转义序列, 其后紧跟另一个转义序列或字面量的结尾, 因此不会延续到其后的字符中。	自动。
Rule 4.2 建议	不使用三字符组, 例如用 `??=` 表示 `#`。	自动。

4.7 标识符

表 4.6 规则 5.1 至 5.9

规范	要求什么	验证方式
Rule 5.1 必需	外部名字在前 31 个字符内互不相同, 31 是 C99 承诺链接器会比较的最大长度。	自动。
Rule 5.2 必需	在同一作用域和名字空间中声明的两个名字, 在前 63 个字符内互不相同, 63 是 C99 承诺编译器会比较的最大长度。	自动。

续页

coding-guidelines-zh.tex Page 26 November 8, 2026 13:15

表 4.6 规则 5.1 至 5.9(续)

规范	要求什么	验证方式
Rule 5.3 必需	块内声明的名字不遮蔽外层作用域中拼写相同的名字。	自动。
Rule 5.4 必需	宏名彼此在前 63 个字符内互不相同, 也与宏参数名不同;同一个宏的参数彼此不同。	自动。
Rule 5.5 必需	用作宏的名字不再用于其它任何东西, 例如对象或函数。	自动。
Rule 5.6 必需	`typedef` 名只用于该 `typedef`, 在程序中不用于任何其它东西。	自动。
Rule 5.7 必需	结构体、联合体或枚举的标签只用于该标签, 在程序中不用于任何其它东西。	自动。
Rule 5.8 必需	具有外部链接的对象或函数的名字在整个程序中不用于其它任何东西, 哪怕是另一个文件中的局部变量。	自动。
Rule 5.9 建议	具有内部链接的对象或函数的名字在程序中不用于其它任何东西, 哪怕在另一个文件中。	自动。

4.8 位域

表 4.7 规则 6.1 至 6.3

规范	要求什么	验证方式
Rule 6.1 必需	位域声明为 `unsigned int`、`signed int` 或 `_Bool`, 或声明为编译器有文档说明的其它显式有符号或无符号整型, 绝不用裸 `int` 或枚举类型。	自动。
Rule 6.2 必需	宽度为一位的具名位域不是有符号类型。	自动。
Rule 6.3 必需	联合体的成员不是位域。	自动。 第 *3*、*4* 号修正案。

coding-guidelines-zh.tex Page 27 November 8, 2026 13:15

4.9 字面量与常量

表 4.8 规则 7.1 至 7.6

规范	要求什么	验证方式
Rule 7.1 必需	不使用以 0 开头的八进制常量, 0 本身除外。	自动。
Rule 7.2 必需	类型为无符号的整型常量用 u 或 U 后缀书写。	自动。
Rule 7.3 必需	常量的后缀用大写 L, 绝不用小写 l, 因为它看起来像数字 1。	自动。
Rule 7.4 必需	字符串字面量只赋给指向 const char 的指针, 因此无法通过它写入。	自动。
Rule 7.5 强制	整型常量宏如 UINT32_C 的参数是不带后缀的普通整型常量, 且其值能容纳在该宏名字所给出的宽度之内。	自动。 第 *3*、*4* 号修正案。
Rule 7.6 必需	不使用窄于 int 的类型的整型常量宏，即 INT8_C、INT16_C、UINT8_C 和 UINT16_C: 它们给出的值具有 int 类型。	自动。 第 *3*、*4* 号修正案。

4.10 声明与定义

表 4.9 规则 8.1 至 8.17

规范	要求什么	验证方式
Rule 8.1 必需	每条声明都写出类型: 没有一条依赖省略类型即为 int 的规则。	自动。
Rule 8.2 必需	函数的类型写成原型, 其中每个参数都有名字。	自动。
Rule 8.3 必需	对象或函数的每次声明, 其参数名以及类型名和限定符都彼此相同。	自动。

续页

coding-guidelines-zh.tex Page 28 November 8, 2026 13:15

表 4.9 规则 8.1 至 8.17(续)

规范	要求什么	验证方式
Rule 8.4 必需	具有外部链接的对象或函数在其定义处能看到一份兼容的声明, 通常来自它的头文件。	自动。
Rule 8.5 必需	外部对象或函数只声明一次, 在一个文件中, 通常是头文件。	自动。
Rule 8.6 必需	具有外部链接的标识符在整个程序中恰好有一个定义。	自动。
Rule 8.7 建议	只有一个文件使用的函数或对象不给予外部链接: 在该文件中声明为 `static`。	自动。
Rule 8.8 必需	具有内部链接的对象或函数的每次声明都写 `static`, 不只第一次。	自动。
Rule 8.9 建议	只有一个函数使用的对象在该函数内部声明, 而不是在文件作用域。	自动。
Rule 8.10 必需	`inline` 函数声明为 `static`。	自动。
Rule 8.11 建议	具有外部链接的数组在声明时写明其大小。	自动。
Rule 8.12 必需	枚举中, 值由省略 = 而隐含的常量, 不会与另一个常量取相同的值。	自动。
Rule 8.13 建议	指针只要没有通过它写入, 就指向 `const` 限定的类型。	自动。 部分。
Rule 8.14 必需	不使用 `restrict` 限定符。	自动。
Rule 8.15 必需	只要对象的某次声明给出了对齐, 该对象的每次声明都给出相同的对齐, 并以相同方式书写。	在 C99 中不会出现。 第 *3*、*4* 号修正案。
Rule 8.16 建议	对象的声明不要求零对齐。	在 C99 中不会出现。 第 *3*、*4* 号修正案。
Rule 8.17 建议	对象的声明至多有一个对齐说明符。	在 C99 中不会出现。 第 *3*、*4* 号修正案。

4.11 初始化

coding-guidelines-zh.tex Page 29 November 8, 2026 13:15

表 4.10 规则 9.1 至 9.7

规范	要求什么	验证方式
Rule 9.1 强制	自动对象在任何读取之前先被赋值。	自动。 部分。
Rule 9.2 必需	数组、结构体或联合体的初始化器写在花括号中，其中嵌套的每个数组、结构体或联合体也各有一对花括号。	自动。
Rule 9.3 必需	数组要么完整初始化，要么用 `{ 0 }` 全部置零，要么只用指定初始化器。	自动。
Rule 9.4 必需	同一个初始化器中没有对象元素被赋值两次。	自动。
Rule 9.5 必需	用指派符初始化的数组写明其大小，使大小不取决于最高的指派符。	自动。
Rule 9.6 必需	使用链式指派符（如 `.header.type`）的初始化器，为它列出的每个值都给出指派符。	自动。 第 *3*、*4* 号修正案。
Rule 9.7 强制	原子对象在其声明中或用 `atomic_init` 先取得第一个值，之后才被其它东西使用。	在 C99 中不会出现。 第 *3*、*4* 号修正案。

4.12 本质类型

表 4.11 规则 10.1 至 10.8

规范	要求什么	验证方式
Rule 10.1 必需	每个操作数的本质类型与其运算符相称：不对布尔值或字符值做算术运算，不对有符号值做按位或移位运算，不对数值使用布尔运算符。	自动。
Rule 10.2 必需	字符只与数值相加，或从字符中减去数值或另一个字符；一个字符加另一个字符、或从数值中减去字符，是没有意义的。	自动。
Rule 10.3 必需	值不赋给本质类型更窄、或本质类型类别不同的对象，例如将有符号赋给无符号。实参和返回值也是如此。	自动。

续页

coding-guidelines-zh.tex Page 30 November 8, 2026 13:15

表 4.11 规则 10.1 至 10.8（续）

规范	要求什么	验证方式
Rule 10.4 必需	把两个操作数带到共同类型的运算符（如 + 或 <），其两个操作数属于同一本质类型类别，例如都是无符号。	自动。
Rule 10.5 建议	强制转换不把值转换成它本不属于的本质类型，例如把数值转成 `bool`，或把数值转成枚举。	自动。
Rule 10.6 必需	运算结果不直接赋给本质类型更宽的对象：先把操作数加宽，使运算本身在更宽的类型中进行。	自动。
Rule 10.7 必需	当某运算的结果又是另一个运算符的操作数时，另一个操作数不具有更宽的本质类型：先把内层运算加宽。	自动。
Rule 10.8 必需	运算结果不强制转换成更宽的本质类型或另一种本质类型类别：改为转换操作数。	自动。

4.13 指针转换

表 4.12 规则 11.1 至 11.10

规范	要求什么	验证方式
Rule 11.1 必需	指向函数的指针不转换成任何其它类型，其它东西也不转换成指向函数的指针。	自动。
Rule 11.2 必需	指向不完整类型（其内容在本文件中看不到）的指针不转换成其它类型，也不从其它类型转换而来。	自动。
Rule 11.3 必需	指向一种对象类型的指针不转换成指向另一种对象类型的指针，但为读取对象的字节而转成指向字符类型的指针除外。	自动。
Rule 11.4 建议	指向对象的指针不转换成整数，整数也不转换成指向对象的指针。	自动。

续页

coding-guidelines-zh.tex Page 31 November 8, 2026 13:15

表 4.12 规则 11.1 至 11.10(续)

规范	要求什么	验证方式
Rule 11.5 建议	指向 `void` 的指针不转换成指向对象类型的指针。	自动。
Rule 11.6 必需	强制转换不在指向 `void` 的指针与算术类型之间进行。	自动。
Rule 11.7 必需	强制转换不在指向对象的指针与非整数的算术类型(如 `bool`、字符或浮点类型)之间进行。	自动。
Rule 11.8 必需	任何转换, 无论是否强制, 都不去掉指针所指向对象的 `const`、`volatile` 或 `_Atomic` 限定符。	自动。
Rule 11.9 必需	空指针写成 `NULL`, 绝不用整型常量 0。	自动。
Rule 11.10 必需	`_Atomic` 限定符不用于 `void`。	在 C99 中不会出现。 第 *3*、*4* 号修正案。

4.14 表达式

表 4.13 规则 12.1 至 12.6

规范	要求什么	验证方式
Rule 12.1 建议	表达式混用运算符时, 用括号表明运算顺序, 而不要求读者回忆 C 的优先级表。	自动。
Rule 12.2 必需	移位的位数至少为零, 且小于被移位值的本质类型的位数。	自动。 部分。
Rule 12.3 建议	不使用逗号运算符。	自动。
Rule 12.4 建议	常量表达式的求值不会使无符号值回绕。	自动。
Rule 12.5 强制	`sizeof` 不用于声明为数组的参数, 因为这种参数实际上是指针, `sizeof` 给出的是指针的大小。	自动。

续页

coding-guidelines-zh.tex Page 32 November 8, 2026 13:15

表 4.13 规则 12.1 至 12.6(续)

规范	要求什么	验证方式
Rule 12.6 必需	不用 . 或 -> 直接访问原子结构体或联合体的成员:整个对象用原子函数或 = 读写。	在 C99 中不会出现。 第 *3*、*4* 号修正案。

4.15 副作用

表 4.14 规则 13.1 至 13.6

规范	要求什么	验证方式
Rule 13.1 必需	初始化器列表中的值没有持久副作用, 例如改变状态的调用, 因为 C 不规定它们的求值顺序。	自动。 部分。
Rule 13.2 必需	表达式在 C 允许的各部分求值顺序下、以及程序线程任意交错执行时, 都给出相同的值, 并留下相同的持久效果。	自动。 部分。
Rule 13.3 建议	使用 ++ 或 -- 的表达式没有其它副作用, 如赋值或改变状态的调用。	自动。
Rule 13.4 建议	赋值的结果不用作值, 例如在条件中或另一个赋值中。	自动。
Rule 13.5 必需	&& 或 || 的右操作数没有持久副作用, 因为只有当左操作数尚未决定结果时它才被求值。	自动。 部分。
Rule 13.6 必需	`sizeof` 的操作数中不含任何可能有副作用的东西, 因为该操作数根本不求值。	自动。

4.16 控制表达式

coding-guidelines-zh.tex Page 33 November 8, 2026 13:15

表 4.15 规则 14.1 至 14.4

规范	要求什么	验证方式
Rule 14.1 必需	循环计数器不具有本质上的浮点类型。	自动。 部分。
Rule 14.2 必需	`for` 循环采用简单形式: 一个循环计数器, 在第一子句中设置, 在第二子句中检验且无副作用, 只在第三子句中修改, 绝不在循环体中修改。	自动。 部分。
Rule 14.3 必需	控制表达式, 例如 `if` 的条件, 不恒为真也不恒为假, 永远运行的循环所允许的形式除外。	自动。 部分。
Rule 14.4 必需	`if`、`while`、`do` 或 `for` 的条件具有本质上的布尔类型: 数值要写成比较, 而不是直接当作真值或假值来检验。	自动。

4.17 控制流

表 4.16 规则 15.1 至 15.7

规范	要求什么	验证方式
Rule 15.1 建议	不使用 `goto`。	自动。
Rule 15.2 必需	`goto` 只向前跳转, 跳到同一函数中靠后的标号。规则 15.1 主张完全不用 `goto`; 本条适用于仍然使用它的地方。	自动。
Rule 15.3 必需	`goto` 只跳到它所在的块或包围它的块中的标号, 绝不跳进另一个块。规则 15.1 主张完全不用 `goto`; 本条适用于仍然使用它的地方。	自动。
Rule 15.4 建议	一个循环至多由一个 `break` 或 `goto` 结束。	自动。
Rule 15.5 建议	函数只有一个出口, 在它的末尾。	自动。

续页

coding-guidelines-zh.tex Page 34 November 8, 2026 13:15

表 4.16 规则 15.1 至 15.7(续)

规范	要求什么	验证方式
Rule 15.6 必需	每个 `if`、`else`、`while`、`do`、`for` 和 `switch` 的体都是花括号中的复合语句; 只有 `else` 之后可以直接跟另一个 `if`。	自动。
Rule 15.7 必需	`if` 与 `else if` 组成的链以普通的 `else` 结尾。	自动。

4.18 switch 语句

表 4.17 规则 16.1 至 16.7

规范	要求什么	验证方式
Rule 16.1 必需	`switch` 语句具有 MISRA 所述的受限形式: 其体只含 `switch` 子句, 每个子句以 `case` 或 `default` 标号开始、以 `break` 结束, 第一个标号之前什么也没有。	自动。
Rule 16.2 必需	`case` 或 `default` 标号只直接放在其 `switch` 的体中, 绝不放在其中嵌套的块内。	自动。
Rule 16.3 必需	每个 `switch` 子句都以始终执行的 `break` 结束, 因此控制绝不会落入下一个子句。	自动。
Rule 16.4 必需	每个 `switch` 语句都有 `default` 标号。	自动。
Rule 16.5 必需	`default` 标号是其 `switch` 的第一个或最后一个标号。	自动。
Rule 16.6 必需	每个 `switch` 语句至少有两个 `switch` 子句; 少于两个的写成 `if`。	自动。
Rule 16.7 必需	`switch` 的控制表达式不具有本质上的布尔类型。	自动。

4.19 函数

coding-guidelines-zh.tex Page 35 November 8, 2026 13:15

表 4.18 规则 17.1 至 17.13

规范	要求什么	验证方式
Rule 17.1 必需	函数不接收个数可变的参数：不使用 <stdarg.h> 中的任何东西。	自动。
Rule 17.2 必需	没有函数调用自身，无论直接还是经由其它函数。	自动。 部分。
Rule 17.3 强制	每个函数在被调用之前就已声明，因此编译器从不需要为它推测声明。	自动。
Rule 17.4 强制	有返回值的函数，其每一条退出路径都经过给出该值的 return 语句。	自动。
Rule 17.5 必需	参数声明为给定大小的数组时，为它传入的每个实参都是至少有那么多元素的数组。	自动。 部分。
Rule 17.6 强制	数组参数不在其方括号内用 static 声明。	自动。
Rule 17.7 必需	函数的返回值被使用，或者调用被强制转换成 void，以表明它是有意丢弃的。	自动。
Rule 17.8 建议	函数不修改自己的参数；它先把参数复制到局部变量。	自动。 部分。
Rule 17.9 强制	声明为 _Noreturn 的函数，无论走哪条路径都不会返回调用者。	在 C99 中不会出现。 第 *3*、*4* 号修正案。
Rule 17.10 必需	声明为 _Noreturn 的函数，其返回类型是 void。	在 C99 中不会出现。 第 *3*、*4* 号修正案。
Rule 17.11 建议	永远不会返回的函数声明为 _Noreturn。	在 C99 中不会出现。 第 *3*、*4* 号修正案。
Rule 17.12 建议	函数名只用于调用它（带参数列表），或在前面加 & 取其地址。	自动。 第 *3*、*4* 号修正案。
Rule 17.13 必需	函数类型不加 const 或 volatile 之类的类型限定符；限定返回类型或指向该函数的指针则无妨。	自动。

4.20 指针与数组

coding-guidelines-zh.tex Page 36 November 8, 2026 13:15

表 4.19 规则 18.1 至 18.10

规范	要求什么	验证方式
Rule 18.1 必需	指针运算和数组下标不超出指针起初所在的数组, 至多到达其末端之后的一个元素。	自动。 部分。
Rule 18.2 必需	只有当两个指针都指向同一数组时, 才从一个指针中减去另一个。	自动。 部分。
Rule 18.3 必需	运算符 <、<=、> 和 >= 只比较指向同一对象的指针。	自动。 部分。
Rule 18.4 建议	不对指针使用 +、−、+= 和 −=: 元素改由下标访问。	自动。
Rule 18.5 建议	一条声明至多有两级指针。	自动。
Rule 18.6 必需	自动或线程局部对象的地址不保存在比该对象存活更久的地方, 例如静态指针或返回值。	自动。 部分。
Rule 18.7 必需	结构体不以柔性数组成员 (以空方括号声明的成员) 结尾。	自动。
Rule 18.8 必需	不使用变长数组, 即大小只在运行时才知道的数组。	自动。
Rule 18.9 必需	临时对象中的数组, 例如函数返回的结构体中的数组, 不转换成指针: 先把该结构体存入具名对象。	自动。 第 *3*、*4* 号修正案。
Rule 18.10 强制	不使用指向变长数组类型的指针, 即大小只在运行时才知道的数组类型。	自动。 第 *3*、*4* 号修正案。

4.21 重叠的存储

表 4.20 规则 19.1 至 19.2

规范	要求什么	验证方式
Rule 19.1 强制	对象不赋值或复制到与它重叠的另一个对象。	自动。 部分。

续页

coding-guidelines-zh.tex Page 37 November 8, 2026 13:15

表 4.20 规则 19.1 至 19.2(续)

规范	要求什么	验证方式
Rule 19.2 建议	不使用 `union` 关键字。	自动。

4.22 预处理器

表 4.21 规则 20.1 至 20.14

规范	要求什么	验证方式
Rule 20.1 建议	`#include` 位于文件中任何代码之前: 只有其它预处理指令和注释在它之前。	自动。
Rule 20.2 必需	`#include` 中的头文件名不含 `'`、`"` 或 `\` 字符, 也不含 `/*` 或 `//`。	自动。
Rule 20.3 必需	`#include` 之后是 `<>` 或 `""` 中的头文件名, 别无其它, 无论按原文还是宏展开之后。	自动。
Rule 20.4 必需	没有宏与关键字同名。	自动。
Rule 20.5 建议	不使用 `#undef`。	自动。
Rule 20.6 必需	宏的实参中不含看起来像预处理指令的行。	自动。
Rule 20.7 必需	宏参数在展开为表达式的地方都加括号, 除非它是 `#` 或 `##` 的操作数。	自动。
Rule 20.8 必需	`#if` 或 `#elif` 的条件求值为 `0` 或 `1`。	自动。
Rule 20.9 必需	`#if` 或 `#elif` 条件中的每个名字都是已定义的宏, `defined` 的操作数除外, 这样未定义的宏不会被默默读成 `0`。	自动。
Rule 20.10 建议	不使用预处理器的 `#` 和 `##` 运算符。	自动。

续页

coding-guidelines-zh.tex Page 38 November 8, 2026 13:15

表 4.21 规则 20.1 至 20.14(续)

规范	要求什么	验证方式
Rule 20.11 必需	跟在 # 之后的宏参数, 其后不再跟 ##。	自动。
Rule 20.12 必需	当实参本身是会在该处展开的宏时, 作为 # 或 ## 操作数的宏参数, 在宏中不再用于其它地方。	自动。
Rule 20.13 必需	以 # 开头的行是真正的预处理指令, 即使在 `#if` 的被跳过的分支中也是如此。	自动。
Rule 20.14 必需	`#else`、`#elif` 或 `#endif` 与它所属于的 `#if`、`#ifdef` 或 `#ifndef` 在同一文件中。	自动。

4.23 标准库

表 4.22 规则 21.1 至 21.26

规范	要求什么	验证方式
Rule 21.1 必需	`#define` 和 `#undef` 不用于 C 保留的名字: 标准宏或库名, 或以一个下划线加一个大写字母开头的名字, 或以两个下划线开头的名字。	自动。
Rule 21.2 必需	没有标识符以 C 保留的名字声明, 例如标准库函数名, 或在文件作用域以一个下划线开头的名字。	自动。
Rule 21.3 必需	不使用 `<stdlib.h>` 中的分配函数, 如 `malloc`、`calloc`、`realloc` 和 `free`。	自动。
Rule 21.4 必需	不使用 `<setjmp.h>` 中的任何东西, 如 `setjmp` 和 `longjmp`。	自动。
Rule 21.5 必需	不使用 `<signal.h>` 中的任何东西。	自动。
Rule 21.6 必需	不使用标准库的输入输出函数, 如 `<stdio.h>` 中的那些。	自动。

续页

coding-guidelines-zh.tex Page 39 November 8, 2026 13:15

表 4.22 规则 21.1 至 21.26(续)

规范	要求什么	验证方式
Rule 21.7 必需	不使用 atof、atoi、atol 和 atoll, 因为它们无法报告不是数字的文本。	自动。
Rule 21.8 必需	不使用 <stdlib.h> 中结束程序的函数: abort、exit、_Exit 和 quick_exit。	自动。
Rule 21.9 必需	不使用 <stdlib.h> 中的 bsearch 和 qsort。	自动。
Rule 21.10 必需	不使用标准库的日期与时间函数, 如 <time.h> 中的那些。	自动。
Rule 21.11 建议	不包含 <tgmath.h>。第 3 号修正案把本条改为建议性, 并为类型通用宏单独立了规则 21.22 和 21.23。	自动。
Rule 21.12 必需	不使用 <fenv.h>(浮点环境)中的任何东西。第 3 号修正案之前, 本条只覆盖其异常标志, 且为建议性。	自动。
Rule 21.13 强制	传给 <ctype.h> 函数的值或是 EOF, 或能放进 unsigned char, 因此裸 char 先转换成 unsigned char。	自动。 部分。
Rule 21.14 必需	不用 memcmp 比较以空字符结尾的字符串, 因为它也会比较结尾之后的字节。	自动。 部分。
Rule 21.15 必需	传给 memcpy、memmove 或 memcmp 的两个指针指向同一类型, const 和 volatile 除外。	自动。
Rule 21.16 必需	memcmp 只比较指针、有符号、无符号、布尔或枚举类型的对象: 绝不比较结构体(它们可能含填充字节), 也不比较浮点值或字符串。	自动。
Rule 21.17 强制	<string.h> 的字符串函数绝不读写超出其指针所指向对象末尾的地方。	自动。 部分。
Rule 21.18 强制	传给 <string.h> 函数的大小不超过它所作用的对象。	自动。 部分。
Rule 21.19 强制	localeconv、getenv、setlocale 和 strerror 的返回值一律当作指向 const: 绝不通过它写入。	自动。 部分。

续页

coding-guidelines-zh.tex Page 40 November 8, 2026 13:15

表 4.22 规则 21.1 至 21.26(续)

规范	要求什么	验证方式
Rule 21.20 强制	asctime、 ctime、 gmtime、 localtime、 localeconv、getenv、setlocale 或 strerror 返回的指针, 在同一函数或与之共用存储的函数再次被调用之后, 不再使用。	自动。 部分。
Rule 21.21 必需	不使用 <stdlib.h> 中的 system。	自动。
Rule 21.22 强制	传给 <tgmath.h> 类型通用宏的每个操作数都是有符号、无符号或浮点数, 绝不是字符、布尔或指针; 只有该宏有复数形式时才可为复数。	由规则 21.11 隐含。
Rule 21.23 必需	传给接收多个操作数的 <tgmath.h> 类型通用宏的每个操作数, 在整数提升之后具有相同的标准类型。	由规则 21.11 隐含。
Rule 21.24 必需	不使用 <stdlib.h> 中的 rand 和 srand。	自动。 第 *3*、*4* 号修正案。
Rule 21.25 必需	每个原子操作和栅栏都使用顺序一致的内存顺序 memory_order_seq_cst, 不带 _explicit 的函数本来就用它。	在 C99 中不会出现。 第 *3*、*4* 号修正案。
Rule 21.26 必需	mtx_timedlock 只用于以 mtx_timed 创建、支持超时的互斥量。	在 C99 中不会出现。 第 *3*、*4* 号修正案。

4.24 资源与线程

表 4.23 规则 22.1 至 22.20

规范	要求什么	验证方式
Rule 22.1 必需	从标准库获得的每个资源, 如内存或打开的文件, 在每条路径上都显式释放。	自动。 部分。
Rule 22.2 强制	只释放由标准库函数分配的内存, 且只释放一次。	自动。 部分。

续页

coding-guidelines-zh.tex Page 41 November 8, 2026 13:15

表 4.23 规则 22.1 至 22.20(续)

规范	要求什么	验证方式
Rule 22.3 必需	同一个文件不同时通过两个不同的流以读和写的方式打开。	自动。 部分。
Rule 22.4 强制	不向只以读方式打开的流写入任何东西。	自动。 部分。
Rule 22.5 强制	指向 FILE 的指针绝不解引用:FILE 对象只通过库函数处理。	自动。
Rule 22.6 强制	流关闭之后,不再使用指向其 FILE 的指针。	自动。 部分。
Rule 22.7 必需	EOF 只与可能返回 EOF 的库函数所返回的未改动值比较,绝不与它被转换(例如转换为 char)之后的值比较。	自动。
Rule 22.8 必需	紧接在调用通过 errno 报告错误的库函数之前,errno 置为零。	自动。 部分。
Rule 22.9 必需	调用通过 errno 报告错误的库函数之后,errno 与零比较。	自动。 部分。
Rule 22.10 必需	errno 只在通过 errno 报告错误的库函数调用之后立刻检验,中间没有其它调用。	自动。 部分。
Rule 22.11 必需	已被 join 或 detach 的线程不再被 join 或 detach。	在 C99 中不会出现。 第 *3*、*4* 号修正案。
Rule 22.12 强制	线程、互斥量、条件变量和线程专有存储只通过为它们准备的标准库函数使用,绝不复制或直接读取。	在 C99 中不会出现。 第 *3*、*4* 号修正案。
Rule 22.13 必需	线程、互斥量、条件变量和线程专有存储不是自动对象或线程局部对象:它们具有静态存储,或位于分配的内存中。	在 C99 中不会出现。 第 *3*、*4* 号修正案。
Rule 22.14 强制	互斥量用 mtx_init 初始化,条件变量用 cnd_init 初始化,之后才被使用;mtx_init 要给定 C 定义的互斥量类型之一。	在 C99 中不会出现。 第 *3*、*4* 号修正案。
Rule 22.15 必需	互斥量、条件变量或线程专有存储键,只在使用它的每个线程都结束后才销毁。	在 C99 中不会出现。 第 *3*、*4* 号修正案。

续页

coding-guidelines-zh.tex Page 42 November 8, 2026 13:15

表 4.23 规则 22.1 至 22.20(续)

规范	要求什么	验证方式
Rule 22.16 必需	给互斥量加锁的线程, 在每条路径上都于线程结束之前自己把它解锁。	在 C99 中不会出现。 第 *3*、*4* 号修正案。
Rule 22.17 必需	线程只解开自己加上的锁, 只与自己已加锁的互斥量一起等待条件变量。	在 C99 中不会出现。 第 *3*、*4* 号修正案。
Rule 22.18 必需	线程不对自己已持有的互斥量再次加锁, 除非该互斥量创建为递归的。	在 C99 中不会出现。 第 *3*、*4* 号修正案。
Rule 22.19 必需	条件变量始终只与一个互斥量一起使用。	在 C99 中不会出现。 第 *3*、*4* 号修正案。
Rule 22.20 强制	线程专有存储键用 `tss_create` 创建之后才被使用。	在 C99 中不会出现。 第 *3*、*4* 号修正案。

4.25 泛型选择

表 4.24 规则 23.1 至 23.8

规范	要求什么	验证方式
Rule 23.1 建议	泛型选择 `_Generic` 只出现在类函数宏的展开中, 作用于该宏的一个实参。	在 C99 中不会出现。 第 *3*、*4* 号修正案。
Rule 23.2 必需	不从宏展开的泛型选择, 其控制表达式没有副作用, 连函数调用都没有。	在 C99 中不会出现。 第 *3*、*4* 号修正案。
Rule 23.3 建议	泛型选择除 `default` 之外至少还给出一个类型。	在 C99 中不会出现。 第 *3*、*4* 号修正案。
Rule 23.4 必需	泛型选择所列的类型都可以匹配: 没有一个是 `const`、`volatile` 或原子的, 没有一个是数组、函数, 或就地定义的结构体或联合体。	在 C99 中不会出现。 第 *3*、*4* 号修正案。
Rule 23.5 建议	泛型选择不依赖在别处会隐式发生的指针转换: 本可转换成所列某类型的指针, 不会落到 `default`。	在 C99 中不会出现。 第 *3*、*4* 号修正案。

续页

coding-guidelines-zh.tex Page 43 November 8, 2026 13:15

表 4.24 规则 23.1 至 23.8(续)

规范	要求什么	验证方式
Rule 23.6 必需	泛型选择的控制表达式的本质类型与其标准类型相同, 因此被 C 变成 `int` 的 `uint8_t` 求和结果先转换回来。	在 C99 中不会出现。 第 *3*、*4* 号修正案。
Rule 23.7 建议	基于泛型选择的宏, 无论选中哪个类型, 都只对其实参求值一次。	在 C99 中不会出现。 第 *3*、*4* 号修正案。
Rule 23.8 必需	泛型选择若有 `default` 关联, 它或是第一个, 或是最后一个。	在 C99 中不会出现。 第 *3*、*4* 号修正案。

5

覆盖率一览

本章统计前面各章逐条所说的内容:各规范如何被验证、还有什么留给人工,以及检查已知会漏掉的违规形式。

5.1 按验证方式统计的规范

数字来自第 4 章,一行一行数出。它们是项目所声明的,而不是对其它任何东西的测量。

表 5.1 按验证方式统计的规范

验证方式	规范数
自动检查, 完全	137
自动检查, 部分	47
由另一条规则隐含	2
在 C99 中不会出现	29
仅靠人工	6
全部规范	221

部分验证的规范只计一次,只算在部分之下;在 C99 中不会出现的规范不算作由任何方式验证。因此在 221 条中,检查覆盖 184 条,其余 6 条必须由人工验证。

5.2 已知的漏检

每项检查都证明了它在违反规则的代码上触发、在遵守规则的代码上保持沉默。有些违规形式已知会漏过,列在这里,以便审阅者知道该看哪里。部分验证的规则按定义就在其中。

coding-guidelines-zh.tex Page 46 November 8, 2026 13:15

表 5.2 已知的漏检

规范	违规形式	为何漏掉
Rule 2.2	被覆盖的防御性初值	这里所有的分析都特意不理会常量初值: 那是一种常见的防御性习惯。
Rule 8.7	头文件中的定义, 只在一个文件中使用	头文件中的声明会被算作在另一个文件中的使用, 因此头文件中的定义永远不会被报告。
Rule 17.2	通过指针的递归	要看出这一点, 需要知道函数指针可能持有的值, 而分析只跟踪直接调用。
Rule 17.5	经局部指针访问到的过小对象	要知道指针在调用点持有什么, 需要做这里的检查都不做的值分析。
Rule 17.8	取其地址并经该地址写入的参数	只有值分析才能看出被调用者通过指针写入。
Dir 4.8	头文件中声明的结构体, 从未被查看内部	头文件中的结构体可能有正在读取的代码之外的使用者, 因此这里无法表明它可以被隐藏。
Rule 20.7	被字符串化的参数, 或调用点的括号	即使每次使用都在参数外加了括号, 宏的定义本身仍会被报告, 于是合规的代码也被报告。

5.3 人工审阅什么

没有检查。指令 1.1、3.1、4.2、4.14、4.15 和 5.2 没有任何检查覆盖, 必须由人工审阅。

部分。检查能发现规则 1.3、2.1、2.2、8.13、9.1、12.2、13.1、13.2、13.5、14.1、14.2、14.3、17.2、17.5、17.8、18.1、18.2、18.3、18.6、19.1、21.13、21.14、21.17、21.18、21.19、21.20、22.1、22.2、22.3、22.4、22.6、22.8、22.9 和 22.10, 以及指令 4.1、4.3、4.4、4.5、4.6、4.7、4.8、4.9、4.11、4.12、4.13、5.1 和 5.3 的部分违规, 其余仍由人工审阅。

5.4 只有人工才验证什么

第 1 章和第 2 章中的这些规则没有检查覆盖。审阅必须:

- **偏离**。每个条目的理由和批准人、批准本身, 以及没有任何强制规则被偏离。
- **版权**。声明是正确的, 所引用的文档按标题和章节引用, 来自别处的代码能以两种许可发布, 以及贡献者签署了协议。
- **注释**。每条注释真实且保持最新。

coding-guidelines-zh.tex Page 47 November 8, 2026 13:15

- **排版**。块内各步骤之间的空行，以及代码是否好读。
- **代码放在哪里**。策略在库里，`src/` 和 `include/` 保持扁平，每个 port 文件夹的 `NOTICE`，标识符前缀，以及取自规范的名字。
- **程序**。每个 `main` 周围的守卫、它在测试运行器中的登记，以及每个模块的测试。
- **协议**。行为取自规范、每个 `@file` 块所引用的章节，以及设计笔记。
- **文本**。MISRA 和规范的内容没有被粘贴进项目。
- **协同**。检查已经运行过、发现项已在代码中修正，以及被改动的检查已运行过它自己的测试。